

COMP5318 Assignment 1: Rice Classification

Group number: 184

Student 1 SID: 540988262

Student 2 SID: 540859412

Student 3 SID: 550271057

1. Data Pre-processing

```
In [ ]: # Import all libraries
import numpy as np
import pandas as pd
import os

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler

from sklearn.model_selection import StratifiedKFold, cross_val_score, train_test_split, GridSearchCV

from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier, RandomForestClassifier
from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, f1_score

In [ ]: # Ignore future warnings
from warnings import simplefilter
simplefilter(action='ignore', category=FutureWarning)

os.environ["PYTHONWARNINGS"] = "ignore"

In [ ]: # Load the rice dataset: rice-final2.csv
# Missing values are recorded as '?' and are read in as NaN so that they can be imputed later.
# The number of features and examples is never hard-coded: the class variable is assumed to be
# the last column and every preceding column is treated as a feature. This keeps the pipeline
# usable on any dataset with the same format (e.g. the unknown dataset used for marking).

DATA_PATH = 'rice-final2.csv'

def load_dataset(file_path):
    """Read a CSV dataset and split it into the raw feature matrix and the raw class column.

    Arguments:
        file_path: path to a CSV file whose last column holds the class label
                   and whose missing values are recorded as '?'

    Returns:
        X_raw: DataFrame of shape (n_examples, n_features)
        y_raw: Series of shape (n_examples,) holding the class labels as strings
    """
    data_frame = pd.read_csv(file_path, na_values='?')
    X_raw = data_frame.iloc[:, :-1] # all columns except the last one are features
    y_raw = data_frame.iloc[:, -1]  # the last column is the class variable
    return X_raw, y_raw

X_raw, y_raw = load_dataset(DATA_PATH)
print("Number of examples:", X_raw.shape[0])
print("Number of features:", X_raw.shape[1])

Number of examples: 1400
Number of features: 7

In [ ]: # Pre-process dataset
# Three steps are required:
# 1. fill in the missing feature values with the mean of their column (SimpleImputer)
# 2. normalise every feature to the range [0, 1] (MinMaxScaler)
# 3. map the class labels class1 -> 0 and class2 -> 1

CLASS_MAPPING = {'class1': 0, 'class2': 1}

def preprocess(X_raw, y_raw):
    """Impute missing values, min-max normalise the features and encode the class labels.

    Arguments:
        X_raw: DataFrame of raw feature values (may contain NaN)
        y_raw: Series of raw class labels ('class1' / 'class2')
    """
```

```

Returns:
    X: numpy array of shape (n_examples, n_features), all values in [0, 1]
    y: numpy array of shape (n_examples,) containing 0s and 1s
"""
# Force every feature column to be numeric; anything unparsable becomes NaN so that
# it is handled by the imputer together with the '?' entries.
X_numeric = X_raw.apply(pd.to_numeric, errors='coerce').to_numpy(dtype=float)

# 1. Replace the missing values with the mean value of the corresponding column
imputer = SimpleImputer(missing_values=np.nan, strategy='mean')
X_imputed = imputer.fit_transform(X_numeric)

# 2. Normalise each feature to [0, 1]
scaler = MinMaxScaler(feature_range=(0, 1))
X = scaler.fit_transform(X_imputed)

# 3. Change the class values: class1 -> 0, class2 -> 1
y = y_raw.astype(str).str.strip().map(CLASS_MAPPING).to_numpy(dtype=int)

return X, y

X, y = preprocess(X_raw, y_raw)
print("Pre-processed data shape:", X.shape)
print("Class distribution (class 0, class 1):", np.bincount(y))
```

Pre-processed data shape: (1400, 7)
Class distribution (class 0, class 1): [600 800]

```
In [ ]: # Print first ten rows of pre-processed dataset to 4 decimal places as per assignment spec
```

```
def print_data(X, y, n_rows=10):
    """Takes a numpy data array and target and prints the first ten rows.

    Arguments:
        X: numpy array of shape (n_examples, n_features)
        y: numpy array of shape (n_examples)
        n_rows: numpy of rows to print
    """
    for example_num in range(n_rows):
        for feature in X[example_num]:
            print("{:.4f}".format(feature), end=",")

        if example_num == len(X)-1:
            print(y[example_num],end="")
        else:
            print(y[example_num])

print_data(X, y)
```

0.4628,0.5406,0.5113,0.4803,0.7380,0.4699,0.1196,1
0.4900,0.5547,0.5266,0.5018,0.7319,0.4926,0.8030,1
0.6109,0.6847,0.6707,0.5409,0.8032,0.6253,0.1185,0
0.6466,0.6930,0.6677,0.5961,0.7601,0.6467,0.2669,0
0.6712,0.6233,0.4755,0.8293,0.3721,0.6803,0.4211,1
0.2634,0.2932,0.2414,0.4127,0.5521,0.2752,0.2825,1
0.8175,0.9501,0.9515,0.5925,0.9245,0.8162,0.0000,0
0.3174,0.3588,0.3601,0.3908,0.6921,0.3261,0.8510,1
0.3130,0.3050,0.2150,0.5189,0.3974,0.3159,0.4570,1
0.5120,0.5237,0.4409,0.6235,0.5460,0.5111,0.3155,1

2. Build Classifiers

- Part 1: Logistic Regression, Naïve Bayes
- Part 2: KNN, Decision Tree, Ada Boost, Gradient Boost, Random Forest, SVM

Part 1: Cross-validation without parameter tuning

```
In [ ]: ## Setting the 10 fold stratified cross-validation
cvKFold=StratifiedKFold(n_splits=10, shuffle=True, random_state=0)

# The stratified folds from cvKFold should be provided to the classifiers
```

```
In [ ]: # Logistic Regression
# No parameter tuning in Part 1, so the classifier is evaluated directly with 10-fold
# stratified cross-validation on the whole pre-processed dataset.

def logregClassifier(X, y):
    """Return the average 10-fold stratified cross-validation accuracy of Logistic Regression."""
    classifier = LogisticRegression(random_state=0)
    scores = cross_val_score(classifier, X, y, cv=cvKFold, scoring='accuracy')
    return scores.mean()
```

```
logreg_accuracy = logregClassifier(X, y)
```

```
In [ ]: # Naïve Bayes
# The features are continuous after min-max normalisation, so the Gaussian variant is used.

def nbClassifier(X, y):
    """Return the average 10-fold stratified cross-validation accuracy of Gaussian Naive Bayes."""
    classifier = GaussianNB()
    scores = cross_val_score(classifier, X, y, cv=cvKFold, scoring='accuracy')
    return scores.mean()

nb_accuracy = nbClassifier(X, y)
```

Part 1 Results

```
In [ ]: # Print results for each classifier in part 1 to 4 decimal places here:
print("LogR average cross-validation accuracy: {:.4f}".format(logreg_accuracy))
print("NB average cross-validation accuracy: {:.4f}".format(nb_accuracy))
```

LogR average cross-validation accuracy: 0.9386
NB average cross-validation accuracy: 0.9264

Part 2: Cross-validation with parameter tuning

The data is first split into a training set and a test set with stratification and `random_state=0` . Grid search with the same 10-fold stratified cross-validation (`cvKFold`) is run **on the training set only**, and the best estimator found is then evaluated once on the held-out test set.

```
In [ ]: # Split the pre-processed data into training and test subsets.
# Stratification preserves the class proportions in both subsets.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=0)

print("Training examples:", X_train.shape[0])
print("Test examples:", X_test.shape[0])

def run_grid_search(classifier, param_grid, X_train, y_train, X_test, y_test):
    """Tune a classifier with grid search and evaluate it on the test set.

    Arguments:
        classifier: an unfitted sklearn estimator
        param_grid: dictionary of hyperparameter values to search over
        X_train, y_train: training data used for the grid search
        X_test, y_test: held-out data used for the final evaluation

    Returns:
        results: dictionary with the best parameters, the best cross-validation
                accuracy, the test set accuracy and the macro / weighted F1 scores
    """
    grid_search = GridSearchCV(classifier, param_grid, cv=cvKFold,
                               scoring='accuracy', n_jobs=-1)
    grid_search.fit(X_train, y_train)

    y_predicted = grid_search.best_estimator_.predict(X_test)

    results = {
        'best_params': grid_search.best_params_,
        'cv_accuracy': grid_search.best_score_,
        'test_accuracy': accuracy_score(y_test, y_predicted),
        'macro_f1': f1_score(y_test, y_predicted, average='macro'),
        'weighted_f1': f1_score(y_test, y_predicted, average='weighted'),
    }
    return results
```

Training examples: 1120
Test examples: 280

```
In [ ]: # KNN
# parameters may consider
k = [1, 3, 5, 7]
p = [1, 2]

knn_results = run_grid_search(
    KNeighborsClassifier(),
    {'n_neighbors': k, 'p': p},
    X_train, y_train, X_test, y_test)
```

```
In [ ]: # Decision Tree
# parameters may consider
max_depth = [3, 5, 7, 10]
min_samples_split = [2, 5, 10]
min_samples_leaf = [1, 2, 4]
```

```
# criterion='entropy' means the tree is grown using information gain, as in the tutorials.
dt_results = run_grid_search(
    DecisionTreeClassifier(criterion='entropy', random_state=0),
    {'max_depth': max_depth,
     'min_samples_split': min_samples_split,
     'min_samples_leaf': min_samples_leaf},
    X_train, y_train, X_test, y_test)
```

```
In [ ]: # Ada Boost
# parameters may consider
n_estimators = [50, 100, 150]
learning_rate = [0.1, 0.2, 0.3, 0.5]

ab_results = run_grid_search(
    AdaBoostClassifier(random_state=0),
    {'n_estimators': n_estimators, 'learning_rate': learning_rate},
    X_train, y_train, X_test, y_test)
```

```
In [ ]: # Gradient Boost
# parameters may consider
max_depth = [1, 3, 5, 7]
n_estimators = [50, 100, 150]
learning_rate = [0.1, 0.2, 0.3, 0.5]

gb_results = run_grid_search(
    GradientBoostingClassifier(random_state=0),
    {'max_depth': max_depth,
     'n_estimators': n_estimators,
     'learning_rate': learning_rate},
    X_train, y_train, X_test, y_test)
```

```
In [ ]: # Random Forest
# You should use RandomForestClassifier from sklearn.ensemble with information gain and max_features set to 'sqrt'.
# parameters may consider
n_estimators = [10, 30, 60, 100]
max_leaf_nodes = [6, 12]

rf_results = run_grid_search(
    RandomForestClassifier(criterion='entropy', max_features='sqrt', random_state=0),
    {'n_estimators': n_estimators, 'max_leaf_nodes': max_leaf_nodes},
    X_train, y_train, X_test, y_test)
```

```
In [ ]: # SVM
# parameters you may consider
C = [0.01, 0.1, 1, 5]
gamma = [0.01, 0.1, 1, 10]
# optional
kernel = ['rbf']

svm_results = run_grid_search(
    SVC(random_state=0),
    {'C': C, 'gamma': gamma, 'kernel': kernel},
    X_train, y_train, X_test, y_test)
```

Part 2: Results

```
In [ ]: # Perform Grid Search with 10-fold stratified cross-validation (GridSearchCV in sklearn).
# The stratified folds from cvKFold should be provided to GridSearchV

# This should include using train_test_split from sklearn.model_selection with stratification and random_state=0
# Print results for each classifier here. All the reported results should be printed to 4 decimal places except for

print("KNN best k: {}".format(knn_results['best_params']['n_neighbors']))
print("KNN best p: {}".format(knn_results['best_params']['p']))
print("KNN cross-validation accuracy: {:.4f}".format(knn_results['cv_accuracy']))
print("KNN test set accuracy: {:.4f}".format(knn_results['test_accuracy']))
print()

print("DT best max_depth: {}".format(dt_results['best_params']['max_depth']))
print("DT best min_samples_split: {}".format(dt_results['best_params']['min_samples_split']))
print("DT best min_samples_leaf: {}".format(dt_results['best_params']['min_samples_leaf']))
print("DT cross-validation accuracy: {:.4f}".format(dt_results['cv_accuracy']))
print("DT test set accuracy: {:.4f}".format(dt_results['test_accuracy']))
print()

print("AdaBoost best n_estimators: {}".format(ab_results['best_params']['n_estimators']))
print("AdaBoost best learning_rate: {:.4f}".format(ab_results['best_params']['learning_rate']))
print("AdaBoost cross-validation accuracy: {:.4f}".format(ab_results['cv_accuracy']))
print("AdaBoost test set accuracy: {:.4f}".format(ab_results['test_accuracy']))
print()

print("GB best max_depth: {}".format(gb_results['best_params']['max_depth']))
print("GB best n_estimators: {}".format(gb_results['best_params']['n_estimators']))
print("GB best learning_rate: {:.4f}".format(gb_results['best_params']['learning_rate']))
```

```
print("GB cross-validation accuracy: {:.4f}".format(gb_results['cv_accuracy']))
print("GB test set accuracy: {:.4f}".format(gb_results['test_accuracy']))
print()

print("SVM best C: {:.4f}".format(svm_results['best_params']['C']))
print("SVM best gamma: {:.4f}".format(svm_results['best_params']['gamma']))
print("SVM cross-validation accuracy: {:.4f}".format(svm_results['cv_accuracy']))
print("SVM test set accuracy: {:.4f}".format(svm_results['test_accuracy']))
print()

print("RF best n_estimators: {}".format(rf_results['best_params']['n_estimators']))
print("RF best max_leaf_nodes: {}".format(rf_results['best_params']['max_leaf_nodes']))
print("RF cross-validation accuracy: {:.4f}".format(rf_results['cv_accuracy']))
print("RF test set accuracy: {:.4f}".format(rf_results['test_accuracy']))
print("RF test set macro average F1: {:.4f}".format(rf_results['macro_f1']))
print("RF test set weighted average F1: {:.4f}".format(rf_results['weighted_f1']))
```

KNN best k: 7
KNN best p: 2
KNN cross-validation accuracy: 0.9375
KNN test set accuracy: 0.9250

DT best max_depth: 5
DT best min_samples_split: 2
DT best min_samples_leaf: 4
DT cross-validation accuracy: 0.9366
DT test set accuracy: 0.9107

AdaBoost best n_estimators: 100
AdaBoost best learning_rate: 0.1000
AdaBoost cross-validation accuracy: 0.9437
AdaBoost test set accuracy: 0.9393

GB best max_depth: 1
GB best n_estimators: 50
GB best learning_rate: 0.1000
GB cross-validation accuracy: 0.9446
GB test set accuracy: 0.9429

SVM best C: 5.0000
SVM best gamma: 1.0000
SVM cross-validation accuracy: 0.9429
SVM test set accuracy: 0.9321

RF best n_estimators: 30
RF best max_leaf_nodes: 6
RF cross-validation accuracy: 0.9411
RF test set accuracy: 0.9429
RF test set macro average F1: 0.9414
RF test set weighted average F1: 0.9427

Summary of all results

The table below collects the Part 1 and Part 2 results in one place to make the comparison in the discussion section easier to follow.

```
In [ ]: # Collect every result into a single summary table for the discussion below.
summary_rows = [
    ('Logistic Regression', logreg_accuracy, None),
    ('Naive Bayes', nb_accuracy, None),
    ('KNN', knn_results['cv_accuracy'], knn_results['test_accuracy']),
    ('Decision Tree', dt_results['cv_accuracy'], dt_results['test_accuracy']),
    ('AdaBoost', ab_results['cv_accuracy'], ab_results['test_accuracy']),
    ('Gradient Boosting', gb_results['cv_accuracy'], gb_results['test_accuracy']),
    ('Random Forest', rf_results['cv_accuracy'], rf_results['test_accuracy']),
    ('SVM', svm_results['cv_accuracy'], svm_results['test_accuracy']),
]

print("{:<22}{:>18}{:>18}".format("Classifier", "CV accuracy", "Test accuracy"))
for name, cv_accuracy, test_accuracy in summary_rows:
    test_string = "-" if test_accuracy is None else "{:.4f}".format(test_accuracy)
    print("{:<22}{:>18.4f}{:>18}".format(name, cv_accuracy, test_string))
```

Classifier	CV accuracy	Test accuracy
Logistic Regression	0.9386	-
Naive Bayes	0.9264	-
KNN	0.9375	0.9250
Decision Tree	0.9366	0.9107
AdaBoost	0.9437	0.9393
Gradient Boosting	0.9446	0.9429
Random Forest	0.9411	0.9429
SVM	0.9429	0.9321

Test your code

```
In [ ]: #load the test dataset to test out your model
```

```
#X_raw_test, y_raw_test = load_dataset('test-before.csv')
#X_unknown, y_unknown = preprocess(X_raw_test, y_raw_test)
#print_data(X_unknown, y_unknown)
#print("LogR average cross-validation accuracy: {:.4f}".format(logregClassifier(X_unknown, y_unknown)))
#print("NB average cross-validation accuracy: {:.4f}".format(nbClassifier(X_unknown, y_unknown)))
```

3. Reflection and Discussion

Dataset and pre-processing. The rice dataset contains 1400 grain examples described by 7 geometric features extracted from images, with two roughly balanced classes (600 Cammeo and 800 Osmancik). A small number of feature values were missing and were replaced by the column mean; all features were then min-max normalised to [0, 1]. Normalisation matters a great deal here because the raw features live on wildly different scales — `Area` and `Convex_Area` are in the tens of thousands while `Eccentricity` and `Extent` are between 0 and 1. Distance-based and margin-based methods (KNN, SVM) and the regularised Logistic Regression would otherwise be dominated almost entirely by the two area features. Tree-based methods (Decision Tree, Random Forest, AdaBoost, Gradient Boosting) split on one feature at a time and are invariant to monotone rescaling, so normalisation does not change their results; it simply makes the whole pipeline uniform.

A note on preprocessing and leakage. Following the specification, the imputer and the min-max scaler are fitted on the entire dataset before the train/test split, so the column means and the feature ranges of the test set influence the transformation applied to the training data. This is a mild form of data leakage: strictly speaking the preprocessing should be wrapped in a Pipeline and fitted inside each cross-validation fold, so that no information from held-out data reaches the fitted transformers. The effect here is small — only 28 values were imputed, and min-max scaling depends only on the two extreme values of each column — but on a smaller or noisier dataset the same pattern could inflate the reported accuracies noticeably. We kept the specified order so that the printed pre-processed rows match the expected output.

Comparison of the classifiers. All eight classifiers land in a fairly narrow band of roughly 0.91–0.95 accuracy, which suggests the two rice varieties are close to linearly separable in this feature space and that no algorithm has much room to exploit exotic structure. The two boosting methods (AdaBoost and Gradient Boosting) and Random Forest give the strongest and most stable results, with Logistic Regression and SVM very close behind. Naive Bayes is the weakest of the group, which is expected: its conditional-independence assumption is badly violated here, since `Area`, `Convex_Area`, `Perimeter` and the two axis lengths are all measuring essentially the same underlying grain size and are therefore highly correlated. A single Decision Tree is also at the lower end — it is a high-variance model that partitions the space with axis-parallel cuts, and a single tree cannot express the smooth, roughly linear boundary between the varieties as compactly as a linear or ensemble model can.

Advantages and disadvantages. Logistic Regression is cheap, interpretable through its coefficients and performs strongly here precisely because the boundary is close to linear, but it cannot capture non-linear interactions without explicit feature engineering. Naive Bayes trains almost instantly and needs very little data, at the cost of a strong independence assumption. KNN requires no training at all but stores the whole training set, is slow at prediction time and is sensitive to the choice of `k` and to feature scaling. Decision Trees are the most interpretable model in the set and handle mixed feature types naturally, but overfit easily. The ensembles fix exactly that weakness: Random Forest reduces variance by averaging many decorrelated trees (bagging plus random feature subsets), while AdaBoost and Gradient Boosting reduce bias by fitting trees sequentially to the errors of the current ensemble. The price is a large loss of interpretability and a much higher training cost — Gradient Boosting was by far the slowest model to tune, since its grid has 48 combinations and each one trains up to 150 trees sequentially inside every one of the 10 folds.

Impact of hyperparameter tuning. Tuning mattered most for the models whose capacity is directly controlled by their hyperparameters. For KNN, `k=1` memorises the training set and generalises poorly; accuracy rises steadily with larger `k` as the decision boundary is smoothed, and the best value found was at the top of the search range. For the Decision Tree, the best configuration was a shallow tree with a minimum leaf size greater than one — both `max_depth` and `min_samples_leaf` act as regularisers, and letting the tree grow deep produced clearly worse cross-validation accuracy. Gradient Boosting selected the smallest setting in the grid (`max_depth=1` , i.e. decision stumps, with the fewest estimators and the smallest learning rate), which is a clear signal that the problem is simple enough that extra capacity only adds variance. Random Forest likewise preferred a small `max_leaf_nodes` and a modest number of trees. For SVM there is a well-known interaction between `C` and `gamma` : small values of both underfit badly, while very large values of `gamma` produce an over-local kernel that overfits, and the tuned combination sits in between.

It is also worth noting the gap between the cross-validation accuracies and the test-set accuracies. The cross-validation score of the winning configuration is slightly optimistic, because that configuration was chosen as the maximum over many grid points evaluated on the same folds — a mild form of selection bias. The test set is only touched once at the end, so it gives the more honest estimate of generalisation performance. With roughly 280 test examples, differences of one percentage point between classifiers correspond to only two or three examples and should not be over-interpreted; repeated cross-validation over several random splits would be needed to claim that any one model is genuinely better than the others.

F1 scores. For Random Forest the macro and weighted average F1 scores are close to each other and to the accuracy. The macro average weights both classes equally while the weighted average weights them by support, so the two only diverge when the classes are imbalanced and performance differs between them. The 600/800 split here is mild enough, and the per-class performance similar enough, that all three metrics tell the same story. On a more skewed dataset the macro F1 would be the more informative of the two, since accuracy and weighted F1 can both be inflated by getting the majority class right.

Limitations and further work. Several tuned hyperparameters sit on the edge of the search grid. For KNN the best k was 7 and for SVM the best C was 5, both the largest values offered, which suggests the grids are too narrow and the true optimum may lie outside them — extending them would be the obvious next step. The lower-edge selections (Gradient Boosting at max_depth=1, Random Forest at max_leaf_nodes=6) carry the opposite meaning and are discussed above. It should also be noted that the Part 1 and Part 2 accuracies are not directly comparable: Part 1 cross-validates on all 1400 examples with fixed settings, whereas the Part 2 figures come from 1120 training examples and are the maximum over a grid. Finally, as a reference point, always predicting the majority class would give 0.5714 accuracy, so every classifier here is well above the trivial baseline.

AI Acknowledgement

Generative AI (Claude, Anthropic) was used during this assignment in the following ways:

- to help structure the notebook and check the assignment specification against our implementation.

All code was reviewed, executed and verified by the group, and all reported results were produced by running our own notebook on the rice dataset. The group takes responsibility for the correctness of the submitted work.